Akademia Górniczo-Hutnicza
im. Stanisława Staszica w Krakowie

AGH University of Krakow

When Do Graph Neural Networks Help?

Class 4 — Baseline and Data Check

Michał Broś[1] Michał Burda[1]

[1]Computational Intelligence
Prof. Adrian Horzyk
AGH University of Krakow

01 Project reminder

AGH
www.agh.edu.pl

Research question

When do GNNs beat MLP / XGBoost, and how much edge noise can they survive before losing?

- **H1:** on clean similar graphs GNNs win; after enough fake edges, tabular models catch up.
- **H2:** on mixed graphs, tabular models may win from the start.

Class 4 focus: working baseline + data audit + first reproducible evaluation on a *clean* graph (no edge corruption yet).

02 Data audit — Cora



Planetoid Cora (PyTorch Geometric)

Nodes	2708
Undirected edges	5278
Features	1433 (row-normalised bag-of-words)
Classes	7
Train / val / test	140 / 500 / 1000
Split overlap	**0** (no leakage between masks)

Features scaled to $[0, 1]$; class sizes uneven (largest class ≈ 818 nodes).

03 Why this audit matters

- Same fixed masks for **every** model (fair comparison)
- Preprocessing / normalisation does not peek at test labels
- Overlap check = 0 $\Rightarrow$ no train–test leakage in splits
- Small, fast dataset $\Rightarrow$ easy multi-seed runs on a laptop

Scope for today

Clean Cora only. Edge deletion / fake edges and mixed graphs come next.

04 Models compared



No graph

- MLP (2-layer)
- XGBoost on node features

With graph

- GCN (2-layer)

Fair protocol

- Identical Planetoid train/val/test masks
- 5 seeds: 0–4; report mean $\pm$ std
- MLP/GCN: 200 epochs, Adam, early pick by best val accuracy
- Code: `code/scripts/run_baselines.py`

05 First results — Cora test accuracy

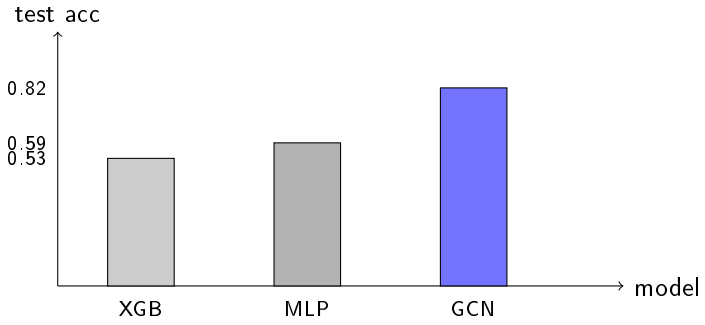
Mean ± std over 5 seeds

Model	Test accuracy	Test macro-F1
XGBoost	0.527 ± 0.006	0.514 ± 0.006
MLP	0.591 ± 0.008	0.572 ± 0.008
GCN	0.818 ± 0.005	0.806 ± 0.005

Saved under `RESULTS/baseline_cora_*.csv` and `data_audit.json`.

06 Clean-graph takeaway

- On clean Cora, **GCN clearly beats** feature-only baselines ($\approx +23$ pp vs MLP, $+29$ pp vs XGBoost).
- Supports the starting point of **H1**: with good (similar) connections, the graph helps.



07 Experiment A — delete real edges



Idea: randomly remove undirected edges: $0\%, 20\%, 40\%, 60\%, 80\%, 100\%$.

- Same Cora splits, 5 seeds, models: MLP / XGBoost / GCN
- Tabular models should stay flat (they ignore edges)
- GCN should fall as structure disappears

08 Experiment A results (Cora)



Test accuracy (mean over 5 seeds)

Model	0%	20%	40%	60%	80%	100%
XGBoost	0.527	0.527	0.527	0.527	0.527	0.527
MLP	0.593	0.592	0.592	0.592	0.592	0.592
GCN	**0.818**	0.793	0.764	0.726	0.673	0.594

Saved in `RESULTS/expA_cora_summary.csv`.

09 Experiment A — conclusions

- MLP / XGBoost are **flat** (as expected).
- GCN drops smoothly: 0.82 → 0.79 (20%) → 0.76 (40%) → 0.73 (60%) → 0.67 (80%) → 0.59 (100%).
- Even after **removing 80%** of edges, GCN still beats MLP ($0.67 > 0.59$).
- At **100%** deletion, GCN ≈ MLP (0.594 vs 0.592).
- Near 30%: between 20% and 40% — GCN still clearly ahead (≈ 0.79–0.76).

Takeaway

On Cora, a lot of graph structure can be removed before GCN loses its advantage. Next: fake edges (Experiment B).

10 Next steps

1. **Experiment B:** add fake edges 0% $\rightarrow$ 75% (hypothesised breaking point $\sim$30%)
2. Add a **mixed** dataset (Roman-empire / filtered Chameleon)
3. Extend models: GraphSAGE, GAT

Class 4 takeaway

Baseline + data audit + Experiment A are done. Graph helps on clean Cora; advantage shrinks as edges are deleted, but survives until nearly all edges are gone.